Deep Q-Network Control of the Unitree G1 Left Elbow

Assignment

CSCN8020 Reinforcement Learning - Assignment 3

Scope

A student-written PyTorch DQN for multi-goal discrete control in the supplied fixed-base Unitree G1 MuJoCo environment.

Outcome

Both controlled epsilon-decay experiments achieved 20/20 successful greedy evaluation episodes. Configuration A (decay 0.995) was selected because it combined 100% success with the highest mean evaluation reward and lowest final absolute error.

Student information

Name: Ce Chen Student ID: 9007166 Date: July 23, 2026

AI assistance disclosure

AI assistance was used to help structure and implement code, automate experiments, generate plots, and draft this report. The student reviewed the submitted components and remains responsible for explaining them. This disclosure should be adjusted to match the course and institutional policy.

1. Introduction and Environment

Primer connection

The G1 Primer established the fixed-base model, left-elbow PD controller, MuJoCo bias-force compensation, Gymnasium interface, success condition, and rule-based validation policy. This assignment replaces only the high-level three-action decision rule with a learned action-value policy.

Observation

The state is [current elbow angle, current elbow velocity, goal angle, goal minus current angle]. Supplying both the goal and signed error allows one network to learn across the full training goal range of -0.8 to +0.8 radians.

Actions

Action 0 decreases the internal controller target, action 1 holds it, and action 2 increases it. The approved low-level controller converts the target into bounded torque while compensating qfrc_bias.

Reward and success

The reward is negative absolute error, plus a success-region bonus and a small penalty for changing the target while close. Success requires error within 0.04 radians for eight consecutive environment steps and adds a terminal bonus of 10.

Termination

A successful episode produces terminated=True. Reaching the 150-step time limit produces truncated=True. Evaluation success is taken only from the true success termination.

2. DQN Architecture and Bellman Update

Network

The online and target networks each use Linear(4,64), ReLU, Linear(64,64), ReLU, and Linear(64,3). No softmax is applied because Q-values are unconstrained estimates of discounted return.

Replay

A bounded deque stores 50,000 transitions. Sampling uses uniformly random batches of 64. Learning begins only after a 500-transition warm-up, reducing correlation and satisfying the required minimum.

Bellman target

For a sampled transition, the selected online value Q(s,a) is compared with y = r + gamma(1-terminated) max_a' Q_target(s',a'), where gamma is 0.95. Targets are calculated under no_grad so gradients cannot flow through the target network.

Truncation treatment

Time-limit truncation does not mask bootstrap because it is an artificial horizon rather than a terminal condition of the robot task. True successful termination does mask bootstrap. Both flags still end the data-collection episode.

Optimization

Adam uses learning rate 0.001 and Smooth L1 (Huber) loss. Gradients are clipped to norm 10. The target network starts as a copy of the online network and is synchronized every 250 optimization steps.

3. Training and Reproducibility

Protocol

Both configurations trained headlessly for 600 episodes on CPU. Python, NumPy, PyTorch, replay sampling, and Gymnasium resets were seeded from 42. Each episode used a deterministic reset seed of 42 plus the zero-based episode index.

Controlled study

Configuration A used epsilon decay 0.995 and Configuration B used 0.985. Both began at 1.0 and were bounded below by 0.05. Gamma, learning rate, batch size, replay capacity, warm-up, network shape, optimizer, target-update interval, goal distribution, episode horizon, and seed policy were identical.

Metrics

The training CSV records episode, seed, sampled goal, cumulative reward, success, length, final error, epsilon, mean episode loss, optimization steps, and elapsed seconds. Checkpoints contain model weights, optimizer state, configuration, dimensions, steps, and experiment metadata.

Commands

Training is run with PYTHONPATH=src python src/train_dqn.py --config-name config_a (or config_b) --episodes 600 --seed 42. Evaluation uses evaluate_dqn.py and epsilon 0.0. Exact commands are preserved in README.md.

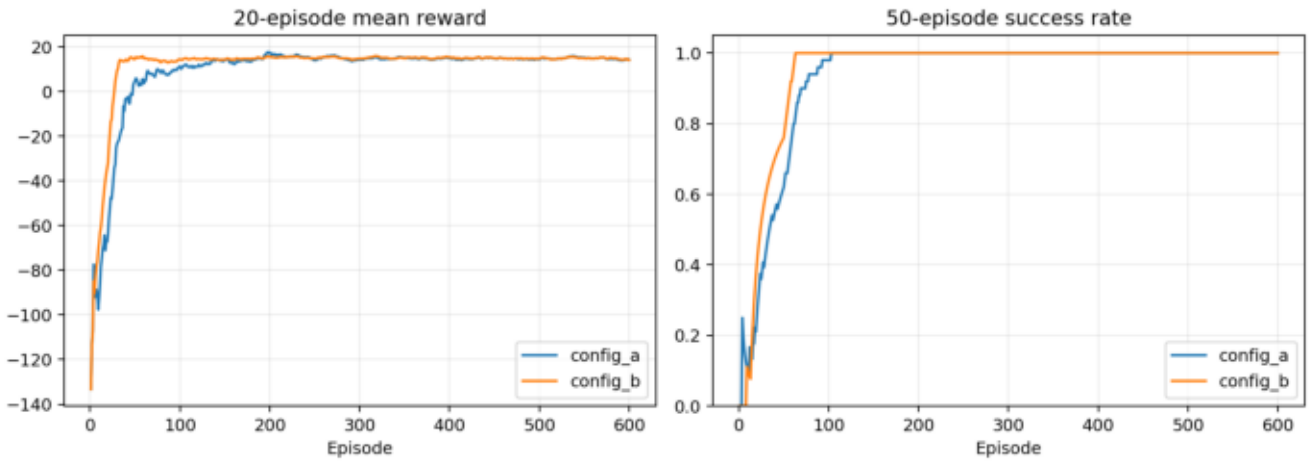
4. Exploration-Decay Study

Config	Decay	Episodes	Seconds	Final eps	Reward-20	Success-50	Greedy eval
config_a	0.995	600	23.73	0.05	14.275	100%	100%
config_b	0.985	600	18.24	0.05	14.194	100%	100%

Interpretation

Both settings converged to a 100% rolling training success rate and 100% greedy evaluation success. B reached minimum exploration much earlier and trained faster, while A retained diverse exploration longer and finished with a slightly higher final-20 mean training reward. The absence of late failures suggests stable learning for both settings.

5. Training Curves



The moving averages show the controlled comparison. Faster decay improves early exploitation, while both curves stabilize at full success. Individual reward, epsilon, loss, and success plots for each configuration are included in the results directories.

6. Selected DQN Evaluation

Goal (rad)	Episodes	Successes	Success rate	Mean reward
-0.8	5	5	100%	11.058
-0.4	5	5	100%	15.447
+0.4	5	5	100%	15.579
+0.8	5	5	100%	11.028
overall	20	20	100%	13.278

Interpretation

Configuration A succeeded in all 20 required greedy episodes, exceeding the 80% threshold. It generalized symmetrically to positive and negative goals. Overall mean reward was 13.278, mean episode length was 19.75, and mean final absolute error was 0.00676 radians. Greedy action counts are retained in the episode-level CSV for HOLD and oscillation analysis.

7. Rule-Based Baseline and Discussion

Policy	Successes	Rate	Mean reward	Mean length	Mean final error
rule_based	20/20	100%	12.867	24.00	0.01221
selected_dqn_config_a	20/20	100%	13.278	19.75	0.00676

Interpretation

Both policies were perfectly successful. The selected DQN was faster on average, earned higher reward, and ended closer to the goal. The rule-based controller is nevertheless far more sample efficient because it requires no training and directly encodes target direction. The DQN learned the same useful pattern: move the controller target toward the goal, then use HOLD so the physical joint can settle. Remaining limitations include evaluation on only four benchmark goals, one training seed, simulation-only evidence, and sensitivity to reward shaping. Future work should repeat multiple seeds, test denser unseen goals, study Double DQN or prioritized replay, and quantify action switching near the target. Configuration A is recommended because its equal success, higher reward, and lower final error outweigh B's modest speed advantage.